

CS 233 OBJECT ORIENTED PROGRAMMING AND DESIGN PATTERNS (OOPDP)



Shamama Anwar

Assistant Professor

Department of Computer Science and Engineering

Birla Institute of Technology, Mesra.

shamama@bitmesra.ac.in

9431103880

Module - V

Strings, characters & Files

- Working with the String and StringBuilder class
- Character class
- Tokenizing strings, Regular Expressions
- Files and Streams, Using NIO classes, Sequential file handling
- Object serialization
- JFileChooser
- Introduction to threading
- Introduction to Generics and lambda expressions.

Handling Strings

- String, StringBuffer ([Module 1](#))
- [StringBuilder](#): Java StringBuilder class is used to create mutable (modifiable) String.

➤ Constructors

Constructor	Description
StringBuilder()	It creates an empty String Builder with the initial capacity of 16.
StringBuilder(String str)	It creates a String Builder with the specified string.
StringBuilder(int length)	It creates an empty String Builder with the specified capacity as length.

Handling Strings

Methods

- **append(String s)**: returns a StringBuilder object by appending the specified string with this string. The append() method is overloaded like append(char), append(boolean), append(int), append(float), append(double) etc.
- **insert(int offset, String s)**: It is used to insert the specified string with this string at the specified position. The insert() method is also overloaded
- **replace(int startIndex, int endIndex, String str)**
- **delete(int startIndex, int endIndex)**
- **reverse()**
- **char charAt(int)**

```
StringBuilder sb=new StringBuilder("My ");  
sb.append("Program");  
System.out.println(sb);
```

Tokenizing Strings

- Character Class ([Module 1: Wrapper Classes](#))
- [Tokenizing Strings](#)
 - Tokens are individual words and punctuation marks.
 - Compilers also perform tokenization.
 - They break up statements into individual pieces like keywords, identifiers, operators and other programming-language elements.
 - Tokens are separated from one another by delimiters, typically white-space characters such as space, tab, newline and carriage return.

Tokenizing Strings

```
import java.util.Scanner;
import java.util.StringTokenizer;
public class TokenTest
{ public static void main(String[] args)
  { Scanner scanner = new Scanner(System.in);
    System.out.println("Enter a sentence:");
    String sentence = scanner.nextLine();
    String[] tokens = sentence.split(" ");
    System.out.printf("Number of tokens",tokens.length);
    for (String token : tokens)
      System.out.println(token);
  }
}
```

Regular Expression

- A regular expression is a String that describes a search pattern for matching characters in other Strings.
- Such expressions are useful for validating input and ensuring that data is in a particular format.
- Class String provides several methods for performing regular-expression operations, the simplest of which is the matching operation.
- String **method** matches receives a String that specifies the regular expression and matches the contents of the String object on which it's called to the regular expression. The method returns a boolean indicating whether the match succeeded.
- A regular expression consists of literal characters and special symbols.
- Some predefined character classes:

```
\d: any digit           \D: any nondigit  
\w: any word character  \W: any nonword character  
\s: any white-space character  
\S: any non-whitespace character
```

Regular Expression

Quantifiers

- * Matches zero or more occurrences of the pattern.
- + Matches one or more occurrences of the pattern.
- ? Matches zero or one occurrences of the pattern.
- {n} Matches exactly n occurrences.
- {n,} Matches at least n occurrences.
- {n,m} Matches between n and m (inclusive) occurrences.

```
public class ValidateInput
{
    public static boolean validateFirstName(String firstName)
    {
        return firstName.matches("[A-Z][a-zA-Z]*");
    }

    public static boolean validateLastName(String lastName)
    {
        return lastName.matches("[a-zA-z]+(['-][a-zA-Z]+)*");
    }

    public static boolean validateAddress(String address)
    {
        return address.matches(
            "\\d+\\s+([a-zA-Z]+|[a-zA-Z]+\\s[a-zA-Z]+)");
    }

    public static boolean validateZip(String zip)
    {
        return zip.matches("\\d{5}");
    }
}
```


Regular Expression

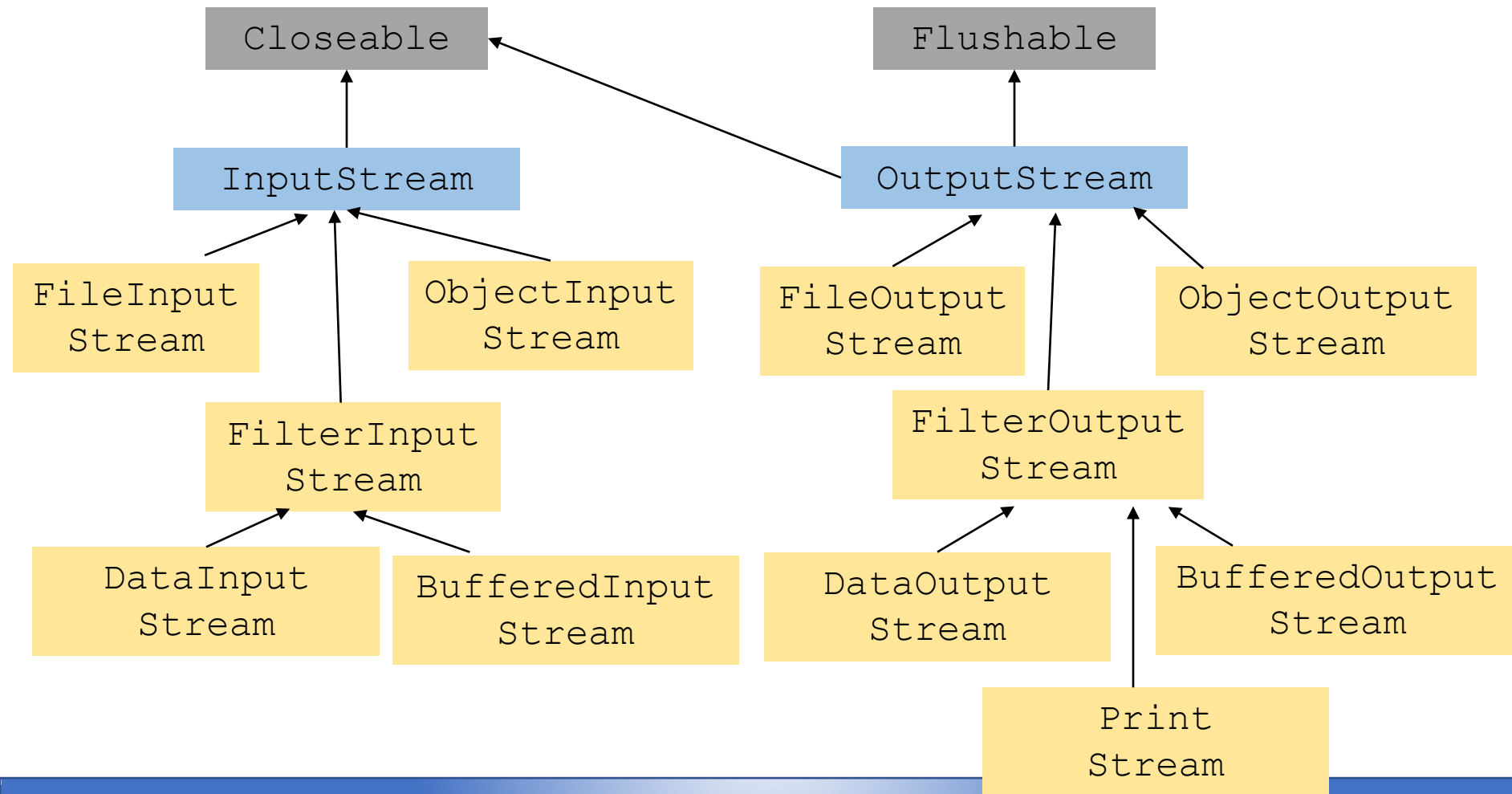
```
import java.util.Scanner;
public class Validate
{ public static void main(String[] args)
  { Scanner scanner = new Scanner(System.in);
    System.out.println("Please enter first name:");
    String firstName = scanner.nextLine();
    System.out.println("Please enter last name:");
    String lastName = scanner.nextLine();
    System.out.println("Please enter address:");
    String address = scanner.nextLine();
    System.out.println("Please enter zip:");
    String zip = scanner.nextLine();
    System.out.println("%nValidate Result:");
    if (!ValidateInput.validateFirstName(firstName))
      System.out.println("Invalid first name");
    else if (!ValidateInput.validateLastName(lastName))
      System.out.println("Invalid last name");
    else if (!ValidateInput.validateAddress(address))
      System.out.println("Invalid address");
    else
      System.out.println("Valid input. Thank you."); } }
```

Files and Streams

- For input / output in Java there are two predefined packages: `java.io` and `java.nio`.
- `java.io` package deals with operations such as reading / writing to console and reading writing to files.
- `java.nio` package contains classes that support the classes in the `java.io` package and perform advanced operations such as buffering, memory mapping, character coding and decoding, pattern matching and locking a file.
- Java I/O facility is based on streams. Stream is a continuous flow of data.
 - Byte stream classes
 - Character stream classes

Byte Stream Class

- Two byte stream abstract classes: `InputStream` and `OutputStream`.



InputStream class

- `int available( )` throws `IOException`: returns the number of available bytes that can be read from an input stream.
- `void close( )` throws `IOException`: closes the input stream.
- `void mark(int readlimit)`: makes a mark at the current position in the input stream. `readlimit` defines after reading how many bytes this mark is nullified
- `boolean markSupported( )`: `mark( )` method works if this method returns true.
- `abstract int read( )`: reads the next byte from the input stream. Subclasses provide implementation for this method. It returns the byte read and -1 if EOF encountered.

InputStream class

- `int read(byte[] b)` : reads bytes and stores them in the byte array `b`. it returns the number of bytes read into the array and `-1` if EOF encountered.
- `int read(byte [] b, int off, int len)` : reads bytes and stores them in byte array `b` upto length `(len)` starting from offset `(off)` in `b`.
- `void reset()` : resets the current pointer to the mark set by the `mark()` method only if `readlimit` has not expired.
- `long skip(long n)` : skips the specified number of bytes(`n`) and returns the number of bytes actually skipped.

OutputStream class

- `void close()` : closes the output stream
- `void flush()` : flushes the output stream
- `void write(byte[] b)` : writes the contents of the byte array to output stream
- `void write(byte[] b, int off, int len)` : writes the specified number of bytes (len) to the output stream starting at offset (off) in b.

File class: Example 1

Program to demonstrate methods in File class

```
import java.io.*;
class FileDemo
{ public static void main(String args[])
  { File f = new File(args[0]);
    System... ("File exists:"+f.exists());
    System... ("File can be read:"+f.canRead());
    System... ("File can be written:"+f.canWrite());
    System... ("File can be executed:"+f.canExecute());
    System... ("File name:"+f.getName());
    System... ("Parent of the file:"+f.getParent());
    System... ("Path of the file:"+f.getPath());
    System... ("Hidden file:"+f.isHidden());
    System... ("Length of the file:"+f.length());
    System... ("Last modified time:"+f.lastModified());
    System... ("It is a File:"+f.isFile());
  }
}
```

Reading Data using Byte Stream: Example 2

FileInputStream

Program to read contents of a file using Byte Stream

```
import java.io.*;
class ReadDemo
{public static void main(String args[])throws IOException
    { File f = new File(args[0]);
      byte[] b;
      if(f.exists())
      { FileInputStream f1 = new FileInputStream(f);
        int num = f1.available();
        b = new byte[num];
        int n = f1.read(b);
        String s = new String(b);
        System.out.println("Contents of"+args[0]+"="+s);
        f1.close(); f = null;
      }
      else
      { System.out.println("File does not exist"); }
    } }
```


Writing Data using Byte Stream: Example 3

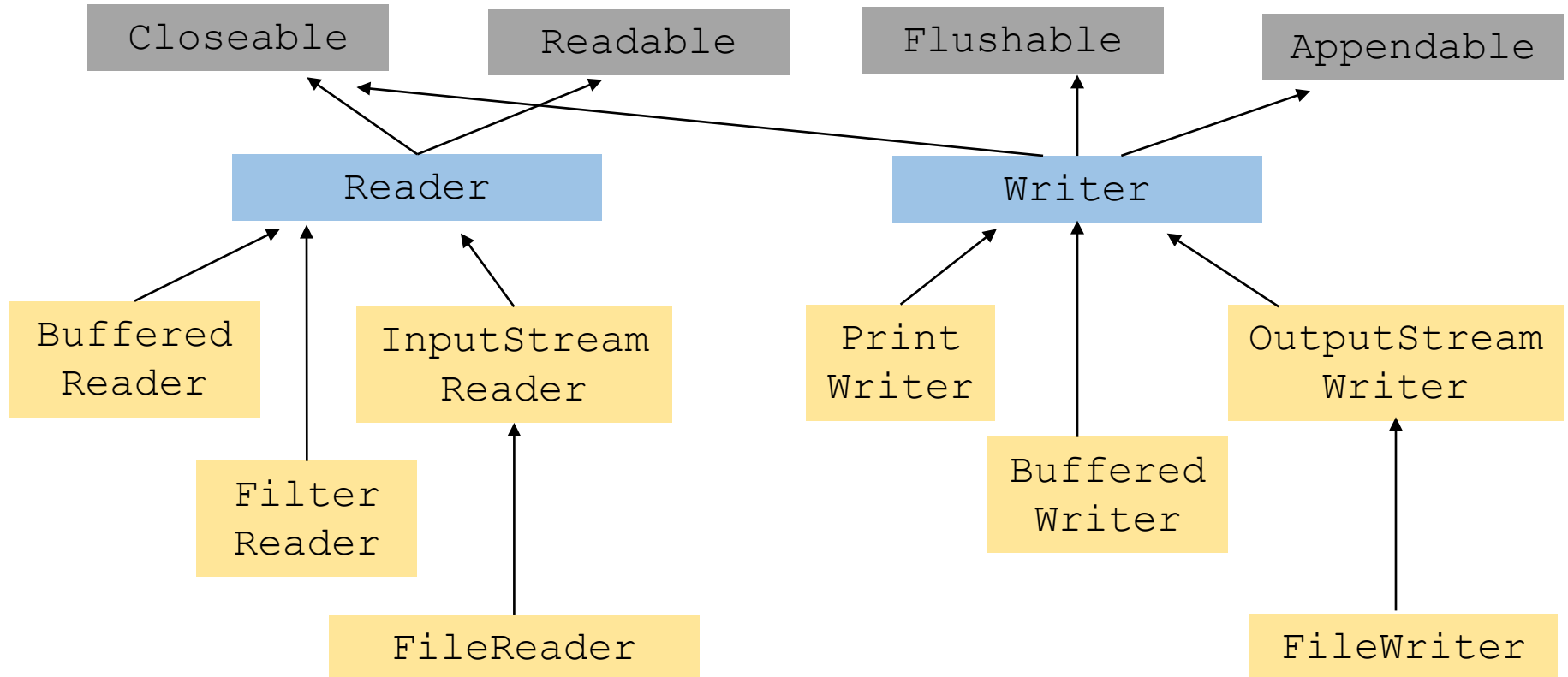
FileOutputStream

Program to write in a file using Byte Stream

```
import java.io.*;
class WriteDemo
{public static void main(String args[])throws IOException
    { File f = new File(args[0]);
      String s = "Welcome to File handling";
      byte[] b = s.getBytes();
      if(f.exists())
      { FileOutputStream f1 = new FileOutputStream(f);
        int num = f1.available();
        f1.write(b);
        f1.close();
        f = null;
      }
      else
      { System.out.println("File does not exist"); }
    } }
```

Character Stream Class

- Two character stream abstract classes: `Reader` and `Writer`.



FileReader / FileWriter

FileReader

- `int read()` : returns a character in ASCII form. Return -1 on EOF
- `void close()` : closes the object of FileReader class.

FileWriter

- `void write(String text)` : It is used to write the string into FileWriter.
- `void write(char c)` : It is used to write the char into FileWriter.
- `void write(char[] c)` : It is used to write char array into FileWriter.
- `void flush()` : It is used to flushes the data of FileWriter.
- `void close()` : It is used to close the FileWriter.

Reading Data using Character Stream: Example 4

FileReader

Program to read contents of a file using Character Stream

```
import java.io.*;
class ReadDemo
{public static void main(String args[])throws IOException
    { File f = new File(args[0]);
      int n;
      if(f.exists())
      { FileReader f1 = new FileReader(f);
        while((n=f1.read()) != -1)
          System...((char)n);
        f1.close();
      }
      else
      { System...("File does not exist"); }
    } }
```

Writing Data using Character Stream: Example 5

FileWriter

Program to write in a file using Character Stream

```
import java.io.*;
class WriteDemo
{public static void main(String args[])throws IOException
    { File f = new File(args[0]);
      String s = "Write Character Stream";
      if(f.exists())
      { FileWriter f1 = new FileWriter(f);
        f1.write(s);
        f1.close();
      }
      else
      { System.out.println("File does not exist"); }
    } }
```

Program

WAP to copy the contents of file “abc.txt” to “xyz.txt” using

- (i) Byte Stream
- (ii) Character Stream

Serialization

- Serialization is the process of converting an object to bytes.
- Deserialization is the process of converting bytes back to an object.
- The class that wants its objects to be serialized needs to implement the `Serializable` interface.
- It is an empty interface (no methods). It is implemented by the class to inform that the objects of that class can be serialized.
- The attributes that we do not want to serialize are made `transient`.
- By default `static` variables are not serialized.

Serialization

- The `ObjectOutputStream` class is used to write primitive data types and Java objects to an `OutputStream`. Only objects that support the `java.io.Serializable` interface can be written to streams.
- **Constructor:**
`ObjectOutputStream(OutputStream out)` throws `IOException`
- **Methods:**
 - `public final void writeObject(Object obj)` throws `IOException`
 - `public void flush()` throws `IOException`
 - `public void close()` throws `IOException`

Deserialization

- The `ObjectInputStream` **deserializes** objects and primitive data written using an `ObjectOutputStream`.
- **Constructor:**
`ObjectInputStream(InputStream out)` throws `IOException`
- **Methods:**
 - `public final Object readObject()` throws `IOException`, `ClassNotFoundException`
 - `public void close()` throws `IOException`

Serialization: Example 15

```
import java.io.*;
class student implements Serializable
{ int id;
  String name;
  String addr;
  int marks;
  Student(int id, String name, String addr, int marks)
  { this.id = id; this.name = name;
    this.addr = addr; this.marks = marks; } }
class SerDemo
{ public static void main(String args[])throws Exception
  { Student s1 =new Student(211,"Rina","Delhi", 340);
    FileOutputStream fout=new FileOutputStream("f.txt");
    ObjectOutputStream out=new ObjectOutputStream(fout);
    out.writeObject(s1);
    out.flush();
    System.out.println("success");
  } }
```

Deserialization: Example 15 contd....

```
class DeDemo
{ public static void main(String args[])throws Exception
{ FileInputStream fin=new FileInputStream("f.txt");
  ObjectInputStream in=new ObjectInputStream(fin);
  Student s=(Student)in.readObject();
  System.out.println("ID: "+s.id);
  System.out.println("Name: "+s.name);
  System.out.println("Address: "+s.addr);
  System.out.println("Marks: "+s.marks);
  in.close();
}
}
```

transient: Example 16

```
import java.io.*;
class student implements Serializable
{ int id;
  String name;
  String addr;
  transient int marks;
  Student(int id, String name, String addr, int marks)
  { this.id = id; this.name = name;
    this.addr = addr; this.marks = marks; } }
class SerDemo
{ public static void main(String args[])throws Exception
  { Student s1 =new Student(211,"Rina","Delhi", 340);
    FileOutputStream fout=new FileOutputStream("f.txt");
    ObjectOutputStream out=new ObjectOutputStream(fout);
    out.writeObject(s1);
    out.flush();
    System.out.println("success");
  } }
```

transient: Example 16 contd.....

```
class DeDemo
{ public static void main(String args[])throws Exception
{ FileInputStream fin=new FileInputStream("f.txt");
  ObjectInputStream in=new ObjectInputStream(fin);
  Student s=(Student)in.readObject();
  System.out.println("ID: "+s.id);
  System.out.println("Name: "+s.name);
  System.out.println("Address: "+s.addr);
  System.out.println("Marks: "+s.marks);
  in.close();
}
}
```

Multithreading

- Multithreading is similar to multi-processing.
- A multi-processing Operating System can run several processes at the same time
 - Each process has its own address/memory space
 - The OS's scheduler decides when each process is executed
 - Only one process is actually executing at any given time. However, the system appears to be running several programs simultaneously
- In a multithreaded application, there are several points of execution within the same memory space.
 - Each point of execution is called a thread
 - Threads share access to memory

Multithreading

- In a single threaded application, one thread of execution must do everything
 - If an application has several tasks to perform, those tasks will be performed when the thread can get to them.
 - A single task which requires a lot of processing can make the entire application appear to be very slow or unresponsive.
- In a multithreaded application, each task can be performed by a separate thread
 - If one thread is executing a long process, it does not make the entire application wait for it to finish.
- If a multithreaded application is being executed on a system that has multiple processors, the OS may execute separate threads simultaneously on separate processors.

Multithreading

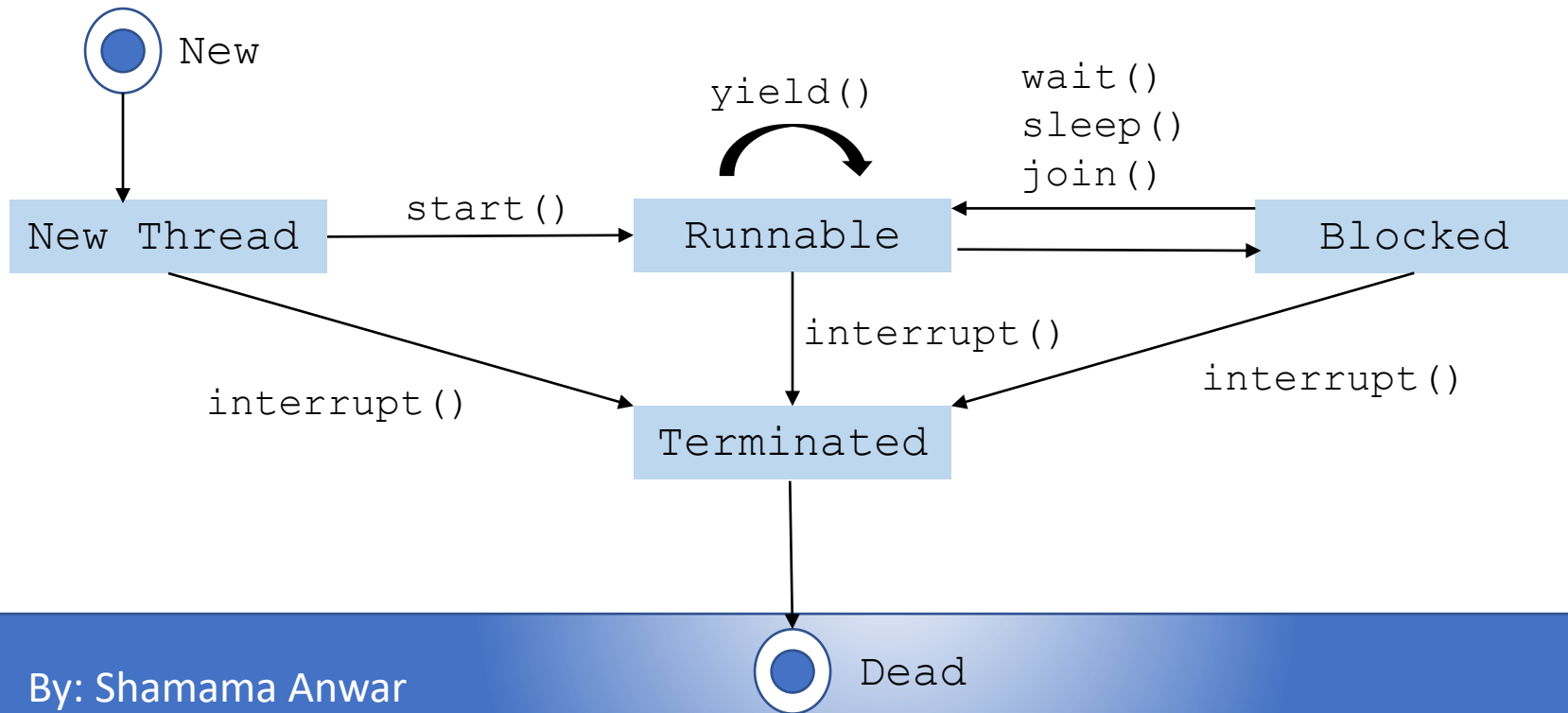
- The Java Virtual machine has its own runtime threads
 - Used for garbage collection
- Threads are represented by a Thread class
 - A thread object maintains the state of the thread
 - It provides control methods such as interrupt, start, sleep, yield, wait
- When an application executes, the main method is executed by a single thread.
 - If the application requires more threads, the application must create them.

Thread

- Threads are lightweight processes as the overhead of switching between threads is less
- They can be easily spawned
- The Java Virtual Machine spawns a thread when your program is run called the Main Thread
- Why do we need threads?
 - To enhance parallel processing
 - To increase response to the user
 - To utilize the idle time of the CPU
 - Prioritize your work depending on priority

Thread State

- Threads can be in one of four states
 - Created, Running, Blocked, and Dead
- A thread's state changes based on:
 - Control methods such as start, sleep, yield, wait, notify
 - Termination of the run method



Creating Thread

- In java threads can be created by extending the `Thread` class or implementing the `Runnable` Interface.
- It is more preferred to implement the `Runnable` Interface if we need to extend properties from other classes as well.
- Implement the `run()` method which is the starting point for thread execution.

Creating Thread using the Thread class

- Constructors in the Thread class
 - Thread()
 - Thread(String name)
 - Thread(Runnable r)
 - Thread(Runnable r, String name)
 - Thread(ThreadGroup g, Runnable r)
- Methods in the Thread class
 - `public void run()` : is used to perform action for a thread.
 - `public void start()` : starts the execution of the thread. JVM calls the `run()` method on the thread.
 - `public static void sleep(long milliseconds)` : Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds.
 - `public void join()` : waits for a thread to die.

Creating Thread using the Thread class

- `public void join(long milliseconds) :` waits for a thread to die for the specified milliseconds.
- `public int getPriority() :` returns the priority of the thread.
- `public int setPriority(int priority) :` changes the priority of the thread.
- `public String getName() :` returns the name of the thread.
- `public void setName(String name) :` changes the name of the thread.
- `public static Thread currentThread() :` returns the reference of currently executing thread.
- `public int getId() :` returns the id of the thread.

Creating Thread using the Thread class

- `public Thread.State getState() :` returns the state of the thread.
- `public boolean isAlive() :` tests if the thread is alive.
- `public static void yield() :` causes the currently executing thread object to temporarily pause and allow other threads to execute.
- `public void suspend() :` is used to suspend the thread.
- `public void resume() :` is used to resume the suspended thread.
- `public void stop() :` is used to stop the thread.
- `public boolean isDaemon() :` tests if the thread is a daemon thread.
- `public void setDaemon(boolean b) :` marks the thread as daemon or user thread.

Creating Thread using the Thread class

- `public void interrupt()` : interrupts the thread.
- `public boolean isInterrupted()` : tests if the thread has been interrupted.
- `public static boolean interrupted()` : tests if the current thread has been interrupted.

```
class Test
{ public static void main(String args[])
  { Thread obj = Thread.currentThread();
    System.out.println("Current Thread:"+obj);
    obj.setName("New Thread");
    System.out.println("Renamed Thread"+obj);
  }
}
```

Current Thread: Thread[main, 5, main]

Renamed Thread: Thread[NewThread, 5, main]

Creating Thread using the Thread class: Ex - 1

```
class Test extends Thread
{
    public void run()
    {System.out.println("thread is running...");
    }
    public static void main(String args[])
    {
        Test t1=new Test();
        t1.start();
    }
}
```


Creating Thread using the Thread class – Ex: 2

```
class Test extends Thread
{
    public void run()
    {
        for(int i=1;i<5;i++)
            System.out.println(i);
    }
}

public static void main(String args[])
{
    Test t1=new Test();
    Test t2=new Test();
    t1.start();
    t2.start();
}
```

Creating Thread using the Runnable interface

- Any class implementing the Runnable interface must override the run() method.
- This can be used if a class requires multiple inheritance.
- Once a class that implements the Runnable interface is created, an object of the Thread class must be instantiated from within that class

Creating Thread using Runnable: Ex - 9

```
class ThreadChild implements Runnable
{ ThreadChild()
    { Thread t = new Thread(this, "Example");
      System.out.println("Details of child thread:"+t);
      t.start();
    }
  public void run()
  { try
    {for(int i=1;i<=5;i++)
      { System.out.println("From child thread 1:"+i);
        Thread.sleep(500);}}
    catch(InterruptedException e)
      {System...("Child thread1 interrupted");}
    System.out...("Exit from child thread 1"); } }
```

Creating Thread using Runnable: Ex – 9 contd...

```
class demo
{ public static void main(String args[])
  { new ThreadChild();
    try
    { for(int m=1; m<=5;m++)
      { System.out.println("From Main Thread:"+m);
        Thread.sleep(1000);
      }
    }
    catch (InterruptedException e)
    { System.out.println("Main interrupted"); }
    System.out.println("Exit from main thread");
  } }
```